Giải thích các biến trong `tinyml_pkg`

Khu lưu biến dùng chung của bộ tăng tốc TinyML trên FPGA (Basys 3 / Nexys A7-100T) · Mạng MLP int8 784-32-10

Package `tinyml_pkg` là nơi duy nhất khai báo các hằng số, độ rộng bit, kiểu dữ liệu và công thức dùng chung. Mọi module khác dùng chúng bằng dòng `import tinyml_pkg::*;` và không bao giờ tự gõ lại các con số như 784 hay 32. Muốn đổi kích thước mạng hay hệ số lượng tử thì chỉ sửa ở một chỗ.

Các biến được chia thành 6 nhóm theo mục đích. Mỗi biến được ghi đủ 5 thông tin: tên, giá trị, cách tính, ý nghĩa (vì sao chọn giá trị đó) và nơi dùng.

1. **Hệ thống và UART**: xung nhịp, tốc độ truyền, thời gian chờ
2. **Kích thước mạng và độ song song**: số đầu vào, số neuron, số MAC
3. **Độ rộng bit dữ liệu**: pixel, activation, trọng số, bias, bộ cộng dồn
4. **Lượng tử hoá**: hệ số requant, giới hạn ReLU
5. **Bộ nhớ**: bộ đệm, ROM trọng số, độ rộng địa chỉ, tên file
6. **Kiểu dữ liệu và hàm dùng chung**

1. Hệ thống và UART

Nhóm này mô tả môi trường mà mạch chạy: tần số xung nhịp của board và cách nhận ảnh từ máy tính qua cổng nối tiếp.

Tên	Giá trị	Cách tính	Ý nghĩa, vì sao chọn	Dùng ở
`CLK_FREQ_HZ`	`100_000_000`	Hằng số	Xung nhịp 100 MHz, tức mỗi chu kỳ 10 ns. Cả Basys 3 và Nexys A7-100T đều có sẵn bộ dao động 100 MHz, nên dùng chung được một giá trị.	UART, file ràng buộc chân (.xdc)
`UART_BAUD`	`115_200`	Hằng số	Tốc độ truyền 115.200 bit/giây, khung 8N1 (1 bit start, 8 bit dữ liệu, 1 bit stop). Đây là tốc độ chuẩn mà mọi phần mềm terminal và thư viện pyserial đều hỗ trợ.	UART RX/TX
`CLKS_PER_BIT`	868	`100.000.000 / 115.200 = 868,05`, phép chia nguyên lấy 868	Số chu kỳ clock cho mỗi bit UART. Bộ thu đếm đủ 868 chu kỳ thì lấy mẫu bit tiếp theo. Sai số do làm tròn chỉ 0,006 %, rất nhỏ so với mức UART cho phép (khoảng 2 %).	UART RX/TX
`GAP_TIMEOUT`	`1_000_000`	`CLK_FREQ_HZ / 100`, tức 10 ms	Nếu giữa 2 byte nghỉ quá 10 ms thì mạch coi như bắt đầu một ảnh mới và đếm lại từ byte 0. Một byte UART chỉ mất khoảng 87 µs, nên 10 ms dài gấp hơn 100 lần, đủ phân biệt "đang gửi" với "đã dừng". Nhờ vậy mạch tự đồng bộ lại nếu bị mất byte.	Khối nhận ảnh ở top

2. Kích thước mạng và độ song song

Nhóm này phải khớp chính xác với mô hình bạn AI huấn luyện. Nếu bạn AI đổi số neuron ẩn, chỉ cần sửa `N_HID`; các độ rộng địa chỉ ở nhóm 5 sẽ tự tính lại.

Tên	Giá trị	Cách tính	Ý nghĩa, vì sao chọn	Dùng ở
`IMG_W, IMG_H`	`28, 28`	Hằng số	Ảnh MNIST có kích thước 28 × 28 pixel.	Tính N_IN
`N_IN`	`784`	`28 × 28`	Số đầu vào của lớp 1, mỗi pixel là một đầu vào. Ảnh được trải phẳng theo hàng: `địa chỉ = hàng × 28 + cột`.	Lớp 1, bộ đệm ảnh
`N_HID`	`32`	Hằng số (bạn AI chọn)	Số neuron lớp ẩn. 32 là mức nhỏ nhất vẫn đạt khoảng 96,5 % độ chính xác, và cả mạng chỉ cần khoảng 200 kbit bộ nhớ. Bản mở rộng trên Nexys A7 có thể tăng lên 128.	Lớp 1 (đầu ra), lớp 2 (đầu vào)
`N_OUT`	`10`	Hằng số	10 lớp đầu ra, ứng với các chữ số 0 đến 9.	Lớp 2, argmax
`PE1`	`8`	Hằng số; `N_HID` phải chia hết cho PE1	Số khối MAC chạy song song ở lớp 1. Mỗi chu kỳ, 8 neuron cùng nhận một pixel và nhân với 8 trọng số khác nhau. Lớp 1 chia thành 32 / 8 = 4 nhóm, mỗi nhóm quét 784 đầu vào, tổng khoảng 3.200 chu kỳ (32 µs). Tăng lên 32 thì nhanh gấp 4 lần nhưng tốn 32 DSP.	Lớp 1, độ rộng ROM w1
`PE2`	`10`	Hằng số; `N_OUT` phải chia hết cho PE2	Số MAC song song ở lớp 2. Chọn bằng đúng N_OUT để tính cả 10 đầu ra trong một lượt (khoảng 45 chu kỳ). Lớp 2 rất nhỏ nên không cần tiết kiệm.	Lớp 2, độ rộng ROM w2

3. Độ rộng bit dữ liệu

Mỗi loại dữ liệu có độ rộng riêng. Chọn quá hẹp thì bị tràn số và sai kết quả; chọn quá rộng thì tốn tài nguyên. Các giá trị dưới đây được tính từ dải giá trị lớn nhất có thể xảy ra.

Tên	Giá trị	Dải giá trị	Ý nghĩa, vì sao chọn	Dùng ở
`PIXEL_W`	8	0 … 255 (không dấu)	Pixel thô từ PC gửi xuống, đúng định dạng ảnh MNIST.	UART, bộ đệm ảnh
`ACT_W`	8	−128 … 127 (có dấu)	Activation int8: đầu vào sau tiền xử lý (0 … 127) và đầu ra lớp ẩn sau ReLU (0 … 127). Dùng kiểu có dấu để nhân trực tiếp với trọng số int8.	Bộ đệm, MAC

WEIGHT_W	8	−127 … 127 (có dấu)	Trọng số int8 sau lượng tử. Không dùng −128 để dải đối xứng quanh 0.	ROM, MAC
BIAS_W	32	int32	Bias được cộng thẳng vào bộ cộng dồn nên phải cùng độ rộng và cùng thang đo với nó.	ROM bias, MAC
ACC_W	32	int32	Bộ cộng dồn. Trường hợp xấu nhất: `784 × 127 × 127 = 12.645.136`, cần 24 bit độ lớn + 1 bit dấu = 25 bit. Chọn 32 bit để có dư và khớp kiểu int32 bên Python.	MAC, requant, logit
PROD_W	16	`ACT_W + WEIGHT_W`	Tích của 2 số 8 bit có dấu cần tối đa 16 bit (ví dụ −128 × −128 = 16.384). Một khối DSP48 làm phép nhân này trong một chu kỳ.	MAC

4. Lượng tử hoá

Sau lớp 1, bộ cộng dồn int32 phải được đưa về int8 trước khi vào lớp 2. FPGA không làm phép chia số thực, nên phép "nhân với một số nhỏ hơn 1" được thay bằng **nhân số nguyên rồi dịch phải**:

```
h = clamp( (acc × M1 + 2^(SHIFT−1)) >>> SHIFT , ACT_MIN , ACT_MAX )
M1 = round( s_w1 × s_x / s_h × 2^SHIFT )        ← bạn AI tính
```

Tên	Giá trị	Cách tính	Ý nghĩa, vì sao chọn	Dùng ở
SHIFT	24	Hằng số	Số bit dịch phải, tương đương chia cho 2^{24} = 16.777.216. Giá trị lớn thì M1 chính xác hơn; 24 bit đủ để M1 giữ khoảng 4 chữ số có nghĩa mà phép nhân vẫn vừa trong 50 bit.	requant_relu
M1	9525	`round(s_w1·s_x/s_h · 2^24)`	Hệ số requant của lớp 1, mã hoá tỉ lệ thực 9525 / 2^{24} ≈ 0,000568. **Đây là giá trị mẫu**, bạn AI sẽ cung cấp số thật sau mỗi lần huấn luyện.	requant_relu
M_W	18	Hằng số	Độ rộng của M1 (có dấu). Quy ước M1 < 2^{17} = 131.072, nên 18 bit là đủ. Hệ số này vừa khít cổng nhân 18 bit của DSP48.	Tính RQ_W
ACT_MAX	127	Giá trị int8 dương lớn nhất	Giới hạn trên (bão hoà): kết quả lớn hơn 127 thì bị cắt về 127, tránh tràn int8.	requant_relu

ACT_MIN	0	Hằng số	Giới hạn dưới, chính là hàm ReLU: kết quả âm bị đưa về 0. Gộp ReLU vào bước clamp nên không tốn thêm phần cứng.	requant_relu
RQ_W	50	`ACC_W + M_W = 32 + 18`	Độ rộng trung gian của tích `acc × M1`. Nhân số 32 bit với số 18 bit cần 50 bit để không bị tràn.	requant_relu

Ví dụ requant với M1 = 9525:
• acc = 10.000 → 10.000 × 9525 + 8.388.608 = 103.638.608 → dịch 24 bit được **6**
• acc = −5.000 → kết quả âm → ReLU đưa về **0**
• acc = 300.000 → khoảng 170 → bão hoà về **127**

5. Bộ nhớ

Nhóm này gồm kích thước các bộ đệm, ROM trọng số và độ rộng địa chỉ. Mọi độ rộng địa chỉ đều được tính tự động bằng `$clog2(n)` (số bit nhỏ nhất để đánh địa chỉ n ô), nên khi đổi kích thước mạng thì địa chỉ tự đúng theo.

5a. Bộ đệm dữ liệu (RAM)

Tên	Giá trị	Cách tính	Ý nghĩa, vì sao chọn	Dùng ở
IMG_DEPTH	784	`N_IN`	Bộ đệm ảnh có 784 ô × 8 bit (khoảng 6,3 kbit), lưu ảnh sau tiền xử lý.	Bộ đệm ảnh
HID_DEPTH	32	`N_HID`	Bộ đệm lớp ẩn có 32 ô × 8 bit, lưu kết quả lớp 1 để lớp 2 đọc.	Bộ đệm ẩn
IMG_ADDR_W	10	`$clog2(784)`	10 bit đánh được 1.024 địa chỉ, đủ cho 784 ô.	Địa chỉ bộ đệm ảnh
HID_ADDR_W	5	`$clog2(32)`	5 bit đánh đúng 32 địa chỉ.	Địa chỉ bộ đệm ẩn
OUT_IDX_W	4	`$clog2(10)`	4 bit biểu diễn chỉ số 0 … 15; chỉ dùng 0 … 9 cho chữ số dự đoán.	Argmax, kết quả

5b. ROM trọng số (BRAM)

Mỗi word ROM chứa trọng số của P neuron tại cùng một đầu vào, để P khối MAC đọc trong một chu kỳ. Bố cục đã chốt với bạn AI:

```
địa chỉ          = g × IN + i          g: nhóm neuron (0 … OUT/P − 1),  i: chỉ số đầu vào
word[8k+7 : 8k]  = W[g × P + k][i]     k = 0 … P − 1, neuron nhỏ nằm ở bit thấp
```

Tên	Giá trị	Cách tính	Ý nghĩa, vì sao chọn	Dùng ở
W1_WORD_W	64	`PE1 × 8 = 8 × 8`	Một word ROM lớp 1 chứa 8 trọng số int8 cho 8 neuron.	ROM w1
W1_DEPTH	3136	`(32 / 8) × 784`	4 nhóm × 784 đầu vào. Tổng ROM lớp 1: 3.136 × 64 = 200.704 bit (khoảng 196 kbit), chiếm khoảng 7 BRAM36.	ROM w1

W1_ADDR_W	12	`$clog2(3136)`	12 bit đánh được 4.096 địa chỉ, đủ cho 3.136 word.	Địa chỉ ROM w1
W2_WORD_W	80	`PE2 × 8 = 10 × 8`	Một word ROM lớp 2 chứa 10 trọng số cho cả 10 neuron đầu ra.	ROM w2
W2_DEPTH	32	`(10 / 10) × 32`	1 nhóm × 32 đầu vào. Tổng ROM lớp 2: 32 × 80 = 2.560 bit, rất nhỏ.	ROM w2
W2_ADDR_W	5	`$clog2(32)`	5 bit cho 32 word.	Địa chỉ ROM w2

5c. Tên file khởi tạo ROM

Tên	Giá trị	Nội dung file	Ý nghĩa	Dùng ở
W1_FILE	`"w1.mem"`	3.136 dòng hex, mỗi dòng 16 ký tự	Do script của bạn AI xuất ra trong thư mục `gen/`. Phần cứng nạp các file này vào ROM bằng lệnh `$readmemh` lúc tổng hợp; trong Vivado cần thêm các file `.mem` vào project. Bias ghi dạng bù 2, ví dụ −1 là `ffffffff`.	ROM trong lớp fully-connected
B1_FILE	`"b1.mem"`	32 dòng, int32 hex		
W2_FILE	`"w2.mem"`	32 dòng hex, mỗi dòng 20 ký tự		
B2_FILE	`"b2.mem"`	10 dòng, int32 hex		

6. Kiểu dữ liệu và hàm dùng chung

6a. Kiểu dữ liệu. `typedef` đặt tên cho một kiểu dữ liệu. Khai báo `act_t x;` rõ nghĩa hơn `logic signed [7:0] x;` và tránh nhầm giữa số có dấu với số không dấu, một lỗi rất hay gặp khi nhân.

Kiểu	Độ rộng	Có dấu?	Dùng cho
pixel_t	8	Không	Pixel thô 0 … 255 từ UART
act_t	8	Có	Activation int8: đầu vào sau tiền xử lý, đầu ra lớp ẩn
weight_t	8	Có	Một trọng số int8
bias_t	32	Có	Một bias int32
acc_t	32	Có	Bộ cộng dồn, logit đầu ra
prod_t	16	Có	Tích int8 × int8 trong MAC
img_addr_t	10	Không	Địa chỉ bộ đệm ảnh
hid_addr_t	5	Không	Địa chỉ bộ đệm ẩn
digit_t	4	Không	Chữ số dự đoán 0 … 9
w1_word_t / w1_addr_t	64 / 12	Không	Word và địa chỉ ROM lớp 1
w2_word_t / w2_addr_t	80 / 5	Không	Word và địa chỉ ROM lớp 2

6b. Hàm dùng chung. Ba hàm này phải cho kết quả **giống hệt từng bit** với mô hình tham chiếu Python của bạn AI.

Hàm	Vào → Ra	Làm gì	Ví dụ
preprocess	`pixel_t → act_t`	Tiền xử lý `x = pixel >> 1`: bỏ bit thấp nhất để đưa 0 … 255 về 0 … 127, vừa với int8 có dấu. Gần như không mất thông tin ảnh.	255 → 127 200 → 100 1 → 0
w1_slice **w2_slice**	`word, k → weight_t`	Lấy trọng số thứ k trong một word ROM, tức các bit `[8k+7 : 8k]`. Dùng toán tử `+:` để chọn 8 bit bắt đầu từ vị trí 8k.	word `0x0102…0780` k = 0 → 0x80 = −128 k = 7 → 0x01 = 1
requant_relu	`acc_t → act_t`	Đưa bộ cộng dồn int32 về int8 theo công thức ở nhóm 4: nhân M1, cộng 2^{23} để làm tròn, dịch phải số học 24 bit (`>>>` giữ dấu cho số âm), rồi kẹp vào [0, 127].	10.000 → 6 −5.000 → 0 300.000 → 127

> **Đã kiểm tra:** hàm `requant_relu` cho kết quả khớp 359/359 giá trị với công thức Python, gồm cả giá trị biên $\pm 2^{31}$. Package biên dịch được bằng Icarus Verilog và qua lint của Verilator (`-Wall`).

Khi nào phải sửa package?

Thay đổi	Biến cần sửa
Bạn AI huấn luyện lại mô hình	`M1` (và các file `.mem`)
Đổi số neuron ẩn (ví dụ 128 trên Nexys A7)	`N_HID`, có thể cả `PE1`
Muốn chạy nhanh hơn	`PE1` (8 → 16 hoặc 32), báo bạn AI xuất lại ROM theo P mới
Đổi tốc độ UART	`UART_BAUD`, đồng thời sửa phía PC